CENTRO DE ENSEÑANZA TÉCNICA Y SUPERIOR

Escuela de ingeniería

Ingeniería en Cibernética Electrónica

Diseño de interfaces

Proyecto final:

CETYS*Parking*

Presentan:

Osiel Gastélum Zazueta, 35596

Rene Lozano García, 32827

Tijuana, B.C., 3 de diciembre de 2025

Índice general

1. Introducción **6**

2. Planteamiento del problema **6**

3. Justificación del problema **8**

4. Objetivos **9**

4.1 Objetivo general 9

4.2 Objetivos particulares 9

5. Desarrollo del proyecto **10**

5.1 Arquitectura del sistema 10

5.1.1 Capa de hardware 10

5.1.2 Capa de Presentación 11

5.2 Implementación de Hardware 12

5.2.1 Microcontrolador STM32F767ZI 12

5.2.2 Distribución de pines GPIO 13

5.2.4 Sistema de Detección con Sensores Inductivos 15

5.2.5 Pantalla OLED SH1106 15

5.2.6 Control de Servomotor 16

5.3 Implementación de Software 16

5.3.1 Comunicación MQTT 16

5.3.2 Lógica de Detección y Cambio de Estado 17

5.4 Integración y Comunicaciones 17

5.4.1 Arquitectura Node-RED 17

5.5 Base de Datos y Almacenamiento 18

5.6 Interfaz de Usuario Web 19

5.6.1 Sistema de Autenticación 20

5.6.2 Vista de Estacionamientos: Monitoreo en Tiempo Real 21

5.6.2.1 Dashboard de Estadísticas Instantáneas 21

5.6.2.2 Grid Visual de Cajones 22
5.6.2.4 Interacción Hardware → Interfaz Web 23
5.6.2.5 Persistencia de estado 24
5.6.2.6 Vista de Reservas: Interacción Web → Hardware 24
5.6.2.6.1 Proceso de Reserva Paso a Paso 24
5.6.2.6.2 Gestión temporal automática 27
5.6.2.7 Vista de Estadísticas: Análisis y Predicción 27
5.6.2.8 Comunicación en tiempo real 29
6. Conclusiones y resultados finales 30
7. Referencias 33
8. Anexos 35
8.1 Anexo A: Fragmento relevante del main() en la STM32 35

Índice de figuras

Figura 1: Arquitectura general del sistema CETYS*Parking* 8
Figura 2: Configuración de pines en CubeMX 11
Figura 3: Diagrama de flujo Node-RED 16
Figura 4: Interfaz web de visualización de estacionamientos 18
Figura 5: Autenticación de inicio de sesión 19
Figura 6: Estadísticas instantáneas 20
Figura 7: Grid visual de estacionamientos con indicadores de color 21
Figura 8: Proceso de reserva, selección de lugar disponible 24
Figura 9: Elección de horario de llegada y salida 24
Figura 10: Confirmación de reserva y visualización de resumen 25
Figura 11: Visualización de estadísticas de disponibilidad y uso 27

Índice de tablas

Tabla 1: Especificaciones técnicas del microcontrolador STM32F767ZI 11

Tabla 2: Distribución de pines para LEDs y sensores inductivos 12

Tabla 3: Estados de cajones y representación mediante colores 13

Tabla 4: Mensajes MQTT implementados en el sistema 16

Tabla 5: Estructura de la base de datos EntradasSalidas 17

CETYS*Parking*

1. Introducción

La gestión eficiente de estacionamientos representa uno de los desafíos más apremiantes en instituciones educativas modernas, donde el crecimiento exponencial de la matrícula estudiantil contrasta con la limitada expansión de la infraestructura vehicular. En el contexto de CETYS Universidad Tijuana, esta problemática se manifiesta en patrones de ocupación impredecibles que generan incertidumbre y pérdida de tiempo para la comunidad universitaria.

El presente proyecto propone el desarrollo de un sistema inteligente de gestión de estacionamientos que integra tecnologías IoT, sensores inductivos y microcontroladores STM32 para optimizar el uso de recursos existentes. A través de una arquitectura basada en detección vehicular en tiempo real y una interfaz web interactiva, el sistema permitirá monitorear la disponibilidad de espacios, gestionar reservas anticipadas y controlar el acceso automatizado mediante actuadores especializados.

La solución implementa una maqueta demostrativa que simula dos estacionamientos con ocho espacios totales, incorporando indicadores LED RGB para señalización visual, sensores inductivos para detección precisa de vehículos, y una pantalla OLED para retroalimentación local. El sistema utiliza Node-RED para la gestión de datos en tiempo real y comunicación MQTT para garantizar la sincronización entre componentes hardware y software.

Esta implementación representa un modelo escalable que puede adaptarse a diferentes contextos institucionales, contribuyendo al desarrollo de infraestructura inteligente y sostenible en el ámbito educativo.

2. Planteamiento del problema

La globalización y urbanización del mundo ha traído consigo problemas constantes problemas en zonas de ciudad debido a la naturaleza de crecimiento exponencial en población de las mismas. Aunque el tráfico de automóviles sugiere ser uno de los aspectos más controversiales, ciudades como la de Tijuana se enfrentan a la escasez de estacionamientos; existe un aumento considerable de espacios de estacionamiento, pero a su vez, un indiscriminado

deseo de usarlos (Hurtado, 2022). En particular, instituciones educativas universitarias se ven afectadas por la irregularidad del número de vehículos registrados a nombre de cada alumno; resulta complicado tener un estimado real de la proporción de alumnos que cuentan con un auto, y, aunado al inevitable crecimiento de la población estudiantil, se evidencia la falta inminente de un cajón de estacionamiento para cada alumno.

La Universidad Autónoma de Baja California registró un crecimiento del 41.49% en su matrícula en Tijuana durante un período de cinco años, una tendencia que se replica en instituciones educativas de la región y que presiona significativamente la infraestructura de estacionamiento existente (Martínez, 2014). Los estándares internacionales recomiendan mantener una proporción de un lugar de estacionamiento por cada 10 estudiantes, sin embargo, la realidad operativa muestra deficiencias en el cumplimiento de esta ratio, especialmente durante horarios pico de clases (Parklio, 2023).

La problemática se intensifica en el campus de Cetys Universidad Tijuana, donde los dos estacionamientos principales experimentan patrones de ocupación impredecibles que generan incertidumbre tanto para estudiantes como para personal administrativo. La ausencia de un sistema de información que permita la gestión inteligente de los recursos existentes constituye la raíz del problema. Las instituciones carecen de herramientas digitales para el monitoreo en tiempo real, análisis de patrones de uso y comunicación proactiva con la comunidad universitaria sobre disponibilidad de espacios. Esta deficiencia tecnológica impide la optimización de los recursos actuales y dificulta la planificación estratégica para el crecimiento institucional proyectado.

La implementación de un sistema de gestión de estacionamientos representa una solución escalable que puede adaptarse al crecimiento de la matrícula estudiantil sin requerir inversiones proporcionales en infraestructura física. El sistema permitirá a los usuarios registrarse para tener acceso y monitorear el flujo vehicular en tiempo real, facilitando la toma de decisiones informadas antes de llegar al campus y optimizando la utilización de ambos estacionamientos disponibles.

3. Justificación del problema

Con la elaboración de este proyecto, se pretende utilizar el microcontrolador STM32 a su máxima capacidad, para sincronizar el control de los periféricos con el correcto funcionamiento. La operación de una página servirá como un gran reto para la implementación de la interfaz para observar el estado de la disponibilidad. Por otro lado, la implementación de los llamados Smart Parking ha demostrado beneficios significativos en diversas instituciones a nivel mundial, estableciendo ecosistemas de servicio especializados que mejoran la experiencia del usuario y la eficiencia operativa (Zimmermann et al., 2023).

La relevancia del proyecto radica en su contribución al desarrollo de ciudades inteligentes mediante la integración de soluciones tecnológicas verdes que fomentan la sustentabilidad ambiental. Los sistemas de información verde (Green IS) han emergido como herramientas fundamentales para abordar desafíos urbanos contemporáneos, donde la gestión eficiente del estacionamiento contribuye directamente a la reducción de emisiones vehiculares y optimización del tráfico urbano (Brauer et al., 2015).

Desde una perspectiva técnica, los avances en sistemas de estacionamiento basados en asignación de recursos y reservas han demostrado su efectividad en la reducción del tiempo de búsqueda y mejora de la satisfacción del usuario. Los sistemas modernos integran sensores IoT, microcontroladores especializados e interfaces web para crear soluciones escalables que pueden adaptarse a diferentes contextos institucionales. La utilización de microcontroladores STM32, sensores inductivos, actuadores LED RGB y sistemas de comunicación ethernet permitirá desarrollar una solución robusta que combine monitoreo en tiempo real con capacidades de reserva anticipada de espacios.

El impacto esperado del proyecto trasciende la solución inmediata del problema de estacionamiento, posicionándose como una inversión estratégica en infraestructura tecnológica que beneficiará a toda la comunidad universitaria. La reducción estimada de 10-15 minutos en tiempo de búsqueda por estudiante

representa no solo mejoras en productividad académica, sino también contribuciones tangibles a la sustentabilidad ambiental mediante la disminución de emisiones vehiculares y consumo de combustible.

4. Objetivos

4.1 Objetivo general

El objetivo general es diseñar y construir un sistema, con apoyo de una maqueta demostrativa, que ejemplifica la operación y contraste los beneficios que supondría la implementación de dicha arquitectura que permite monitorear la disponibilidad de los estacionamientos en Cetys.

4.2 Objetivos particulares

1. **Interfaz web.** Comunicación en tiempo real con Node-RED para mostrar el número de espacios libres; diseño gráfico del estacionamiento, con el asfalto y líneas divisorias de espacios para facilitar la identificación de lugares.
2. **Control de acceso.** Control de pluma con Servo motor al detectar a un carro en la entrada. El acceso se permite o no dependiendo de la disponibilidad de cajones.
3. **Indicador visual.** Mediante una pantalla OLED, mostrar disponibilidad en la entrada, al lado del Servo motor.
4. **Reserva de cajones.** Reserva de espacios a través del interfaz web; se permite un cambio de LED específico a lo largo del tiempo de reserva definido.
5. **Estacionamiento múltiple.** Creación de dos espacios de estacionamiento distintos (dos en total, con ocho cajones cada uno) para permitir su aplicación en instituciones como Cetys Universidad.
6. **Sincronización temporal.** Se monitorean los tiempos de acceso, ocupación de cajones y salida del estacionamiento con respecto al reloj real (Pacific Time).

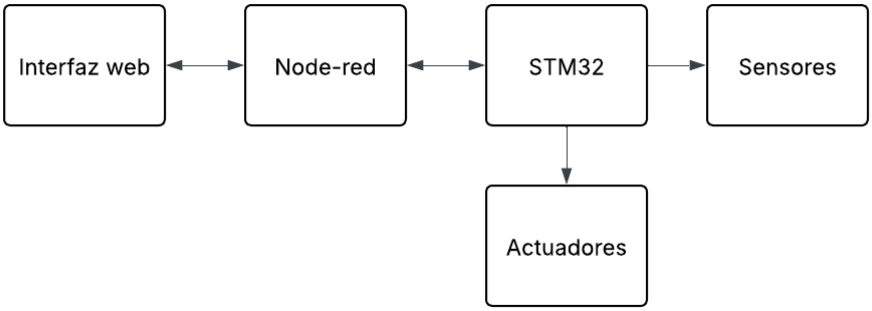
5. Desarrollo del proyecto

5.1 Arquitectura del sistema

El sistema CETYS*Parking* implementa una arquitectura distribuida de tres capas que garantiza modularidad, escalabilidad y mantenibilidad del sistema completo.

Figura 1

Arquitectura general del sistema CETYSParking



5.1.1 Capa de hardware

La capa física del sistema está compuesta por el microcontrolador STM32F767ZI Nucleo-144, que actúa como núcleo central de procesamiento. Este dispositivo fue seleccionado por sus capacidades de conectividad Ethernet RMII, su procesador ARM Cortex-M7 de 216 MHz, y su amplia disponibilidad de periféricos integrados.

Para la detección vehicular, se implementaron ocho sensores inductivos por estacionamiento conectados a pines GPIO configurados como entradas digitales con interrupciones externas (EXTI). Estos sensores proporcionan señales de bajo nivel (0V) cuando detectan la presencia de un vehículo metálico, y alto nivel (3.3V) cuando el espacio está libre.

La señalización visual se implementó mediante LEDs RGB distribuidos en tres arrays independientes por estacionamiento: LEDs verdes (estado libre), LEDs rojos (estado ocupado) y LEDs azules (estado reservado). Esta codificación por colores es consistente con estándares internacionales de señalización.

El control de acceso se implementa mediante un servomotor conectado al timer TIM1, configurado para generar señales PWM con frecuencia de 50 Hz. El servomotor simula una pluma de acceso que se levanta cuando hay cajones disponibles y permanece cerrada cuando el estacionamiento está lleno.

Para retroalimentación local, se integró una pantalla OLED SH1106 de 128x64 píxeles comunicada mediante protocolo SPI. Esta pantalla muestra en tiempo real el número de cajones disponibles, ocupados y reservados.

Capa de Procesamiento y Comunicaciones

El firmware del STM32 fue desarrollado utilizando STM32CubeIDE con la biblioteca HAL de STMicroelectronics. La arquitectura de software implementa un sistema de eventos basado en interrupciones y polling periódico para detección de cambios de estado.

La comunicación con la capa de presentación se implementó mediante MQTT sobre Ethernet, utilizando la biblioteca Mongoose Embedded Web Server Library integrada con mbedTLS para comunicación segura. El microcontrolador actúa como cliente MQTT conectándose a un broker HiveMQ Cloud en la nube, publicando mensajes en el topic "device1/tx" cada vez que ocurre un cambio de estado en algún cajón.

Node-RED actúa como orquestador central del sistema, suscribiéndose a los topics MQTT para recibir eventos desde el hardware, procesando los datos mediante nodos de transformación, almacenando información histórica en MySQL, y redistribuyendo eventos procesados hacia la interfaz web mediante uibuilder y hacia el microcontrolador mediante el topic "device1/rx".

5.1.2 Capa de Presentación

La interfaz web fue desarrollada utilizando Vue.js 3 con arquitectura reactiva, permitiendo actualizaciones instantáneas ante cambios de estado. La aplicación implementa tres vistas principales: visualización de estacionamientos, sistema de reservas, y panel de estadísticas con predicción basada en datos históricos.

La comunicación en tiempo real entre Node-RED y la interfaz web se implementó mediante Socket.IO a través del módulo uibuilder, garantizando

latencia mínima en la actualización de estados visuales. El sistema de autenticación implementa un esquema con credenciales del usuario, con capacidad de extensión hacia sistemas institucionales.

5.2 Implementación de Hardware

5.2.1 Microcontrolador STM32F767ZI

El microcontrolador seleccionado es el STM32F767ZI montado en la placa Nucleo-144, proporcionando un entorno de desarrollo robusto y profesional.

Tabla 1. Especificaciones técnicas del microcontrolador STM32F767ZI

Característica	**Especificación**
Procesador	ARM Cortex-M7 @ 216 MHz
Memoria Flash	2 MB
Memoria RAM	512 KB
Conectividad	Ethernet 10/100 RMII
Perifericos	4x SPI, 4x I2C, 8x USART
Timers	2x 32-bit, 10x 16-bit
ADC	3x 12-bit, 24 canales
GPIO	168 pines I/O

La configuración del microcontrolador se realizó mediante STM32CubeMX, generando el código de inicialización para los siguientes periféricos clave: Ethernet (ETH) en modo RMII, SPI1 para comunicación con pantalla OLED, Timers (TIM1, TIM2, TIM9) para PWM y delays, RTC con cristal LSE para mantenimiento de tiempo preciso, RNG para generación de números aleatorios en conexiones TLS, y múltiples pines GPIO organizados funcionalmente.

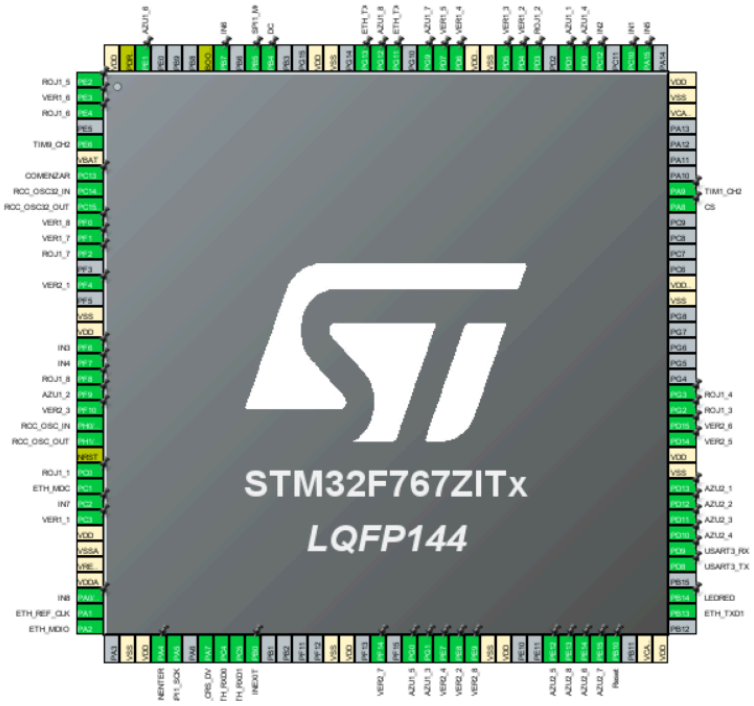
5.2.2 Distribución de pines GPIO

Tabla 2. Distribución de pines para LEDs y sensores inductivos

Función	Cantidad	Configuración
LEDs Verdes (libre)	8 por estacionamiento	Output Push-Pull, Pull down
LEDs Rojos (ocupado)	8 por estacionamiento	Output Push-Pull, Pull down
LEDs Azules (reservado)	8 por estacionamiento	Output Push-Pull, Pull down
Sensores inductivos	4 por estacionamiento	Input, Pull-up, EXTI
Servomotor PWM	2 en total	TIM1 Channel 1, 50 Hz
OLED SPI	5 pines	SP1, CS, DC, RESET

Figura 2

Configuración de pines en CubeMX



5.2.3 Sistema de LEDs RGB

Cada cajón de estacionamiento cuenta con un LED RGB de cátodo común, conectado a tres pines GPIO independientes (uno por color). El sistema implementa control mediante nivel lógico bajo para encendido (LEDON = 0) y nivel alto para apagado (LEDOFF = 1).

Tabla 3. Estados de cajones y representación mediante colores

Estado	**Color LED**	**Valor constante**	**Significado**
Libre	Verde	LIBRE = 0	Cajón disponible
Ocupado	Rojo	OCUPADO = 1	Cajón ocupado
Reservado	Azul	RESERVADO = 2	Cajón reservado

La función de control de LEDs implementa una máquina de estados simple que apaga todos los colores primero y luego enciende el correspondiente al estado deseado:

```
void SetLEDState(int spotIndex, int state) {
   // Apagar todos los colores
   HAL_GPIO_WritePin(leds_verdes[spotIndex].port,
                     leds_verdes[spotIndex].pin, LEDOFF);
   HAL_GPIO_WritePin(leds_rojos[spotIndex].port,
                     leds_rojos[spotIndex].pin, LEDOFF);
   HAL_GPIO_WritePin(leds_azules[spotIndex].port,
                     leds_azules[spotIndex].pin, LEDOFF);
      // Encender el color correspondiente
   switch(state) {
      case LIBRE:
         HAL_GPIO_WritePin(leds_verdes[spotIndex].port,
                           leds_verdes[spotIndex].pin, LEDON);
         break;
      case OCUPADO:
         HAL_GPIO_WritePin(leds_rojos[spotIndex].port,
                           leds_rojos[spotIndex].pin, LEDON);
         break;
      case RESERVADO:
```

```
        HAL_GPIO_WritePin(leds_azules[spotIndex].port,
                    leds_azules[spotIndex].pin, LEDON);
        break;
    }
}
```

5.2.4 Sistema de Detección con Sensores Inductivos

Los sensores inductivos se conectaron a pines GPIO configurados con resistencias pull-up internas y detección de flanco mediante EXTI (External Interrupt). El sistema implementa anti-rebote por software con ventana temporal de 50 ms para evitar falsas detecciones causadas por ruido eléctrico o vibraciones mecánicas.

```
uint32_t lastInterruptTime[8] = {0};
#define DEBOUNCE_DELAY 50 // ms

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    uint32_t currentTime = HAL_GetTick();
    int spotIndex = GetSpotIndexFromPin(GPIO_Pin);

    if(spotIndex >= 0 && spotIndex < 8) {
        if((currentTime - lastInterruptTime[spotIndex]) > DEBOUNCE_DELAY) {
            lastInterruptTime[spotIndex] = currentTime;
            ProcessSensorChange(spotIndex);
        }
    }
}
```

5.2.5 Pantalla OLED SH1106

La pantalla OLED fue integrada mediante la biblioteca sh1106.h, una implementación optimizada para STM32 que permite renderizado de texto y gráficos básicos. La actualización de pantalla se realiza cada segundo en el loop principal, mostrando el título del sistema, estadísticas de ocupación en tiempo real y la hora actual.

5.2.6 Control de Servomotor

El servomotor se controla mediante PWM generado por TIM1, configurado con prescaler y periodo calculados para generar una frecuencia de 50 Hz. Las posiciones se definen mediante duty cycle: 0.5 ms (50 en contador) para pluma cerrada a 0°, y 2.5 ms (250 en contador) para pluma abierta a 180°.

5.3 Implementación de Software

El firmware del STM32 implementa un sistema de eventos basado en interrupciones y polling periódico. La estructura se organiza en módulos especializados para inicialización, comunicación MQTT, gestión de estados y control de actuadores.

5.3.1 Comunicación MQTT

La comunicación MQTT se implementó utilizando la biblioteca Mongoose, que proporciona un cliente MQTT ligero optimizado para sistemas embebidos. El cliente se configura con credenciales seguras, se conecta al broker HiveMQ Cloud mediante TLS/SSL, y mantiene una conexión persistente con keepalive de 60 segundos.

```
struct mg_mqtt_opts opts = {
    .clean = true,
    .qos = 0,
    .retain = false,
    .version = 4,
    .keepalive = 60,
    .client_id = mg_str("STM32_Parking_1"),
    .user = mg_str("cetysparking"),
    .pass = mg_str("***************"),
    .topic = mg_str("device1/tx")
};
```

El sistema utiliza mensajes JSON para comunicación bidireccional. Cada cambio de estado genera un mensaje que incluye identificador de estacionamiento, identificador de cajón, número de cajón legible, estado actual y timestamp Unix en milisegundos.

```
{
   "parkingId": "1",
   "spotId": 3,
   "spotNumber": "PS003",
   "estado": "ocupado",
   "timestamp": 1733270400000
}
```

5.3.2 Lógica de Detección y Cambio de Estado

El sistema implementa una máquina de estados para cada cajón. Cuando un sensor detecta un cambio (vehículo entra o sale), se actualiza el estado interno, se cambia el color del LED correspondiente, se envía notificación MQTT, se actualiza la pantalla OLED y se recalcula el estado de la barrera de acceso.

5.4 Integración y Comunicaciones

5.4.1 Arquitectura Node-RED

Node-RED actúa como el cerebro central del sistema, orquestando la comunicación entre el microcontrolador STM32, la base de datos MySQL y la interfaz web. El flujo implementado consta de varios nodos especializados que realizan funciones específicas de transformación y enrutamiento de datos.

El flujo principal incluye:

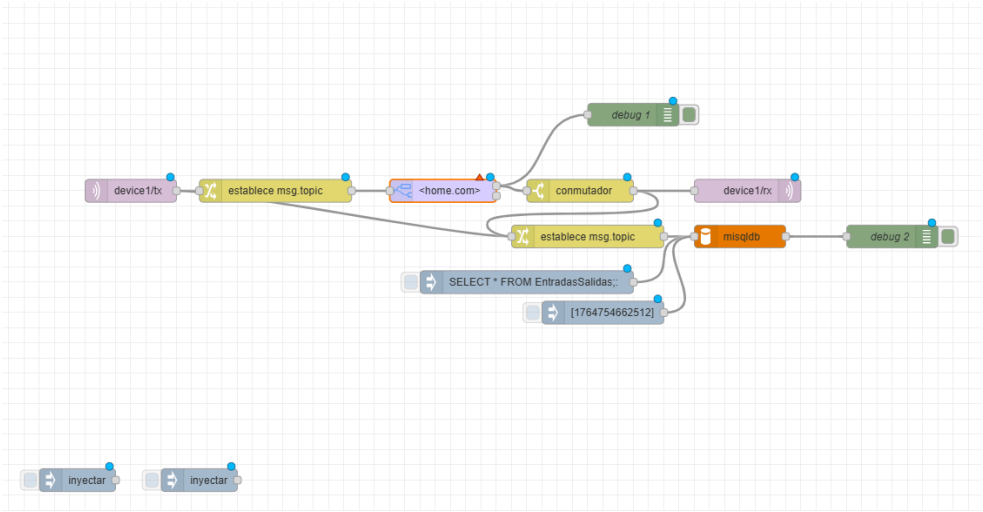
1. Nodo MQTT In: Se suscribe al topic "device1/tx" para recibir eventos del STM32
2. Nodo Change: Transforma el topic a "spot-status-change" para routing interno
3. Nodo UIBuilder: Envía eventos a la interfaz web mediante Socket.IO
4. Nodo Switch: Filtra mensajes por topic para procesamiento selectivo
5. Nodo Change: Prepara query SQL con parámetros nombrados
6. Nodo MySQL: Inserta registro en tabla EntradasSalidas
7. Nodo MQTT Out: Publica respuesta en "device1/rx" para el STM32

Tabla 4. Mensajes MQTT implementados en el sistema

Topic	Dirección	Formato	Propósito
device1/tx	STM32 → Node-RED	JSON	Notificar cambios de estado
device1/rx	Node-RED → STM32	JSON	Actualizar reservas

Figura 3

Diagrama de flujo Node-RED



5.5 Base de Datos y Almacenamiento

El sistema utiliza MySQL como base de datos relacional para almacenamiento persistente de todos los eventos de entrada y salida de vehículos. Esta información histórica permite análisis de patrones de uso, generación de estadísticas y predicción de disponibilidad.

Tabla 5. Estructura de la base de datos EntradasSalidas

Campo	Tipo	Descripción
id	INT AUTO_INCREMENT	Clave primaria, identificador único

parkingId	VARCHAR(2)	ID del estacionamiento (1 o 2)
spotId	INT	ID numérico del cajón (1-8)
spotNumber	VARCHAR(5)	Número legible del cajón (PS001-PS008)
estado	VARCHAR(9)	Estado del cajón (libre/ocupado)
ttimestamp	BIGINT	Timestamp Unix en milisegundos

```
CREATE TABLE EntradasSalidas(
   id INT PRIMARY KEY AUTO_INCREMENT,
   parkingId VARCHAR(2),
   spotId INT,
   spotNumber VARCHAR(5),
   estado VARCHAR(9),
   ttimestamp BIGINT
);
```

Esta estructura permite consultas eficientes para:

- Obtener historial completo de un cajón específico
- Calcular tiempo promedio de ocupación por cajón
- Identificar patrones de uso por horario
- Generar predicciones de disponibilidad basadas en datos históricos
- Analizar utilización comparativa entre estacionamientos

La tabla se actualiza en tiempo real con cada cambio de estado detectado por los sensores, creando un registro completo de la actividad del sistema.

5.6 Interfaz de Usuario Web

La interfaz web fue desarrollada como una Single Page Application (SPA) utilizando Vue.js 3, proporcionando una experiencia de usuario fluida y responsiva. La aplicación se estructura en tres vistas principales accesibles

mediante pestañas de navegación, cada una diseñada para cumplir funciones específicas en la interacción usuario-sistema.

Figura 4

Interfaz web de visualización de estacionamientos

5.6.1 Sistema de Autenticación

El sistema implementa autenticación básica con credenciales de usuario mediante un modal de login que se presenta al iniciar la aplicación. Los usuarios deben ingresar su matrícula institucional y contraseña antes de acceder a cualquier funcionalidad del sistema. Al iniciar sesión correctamente, el usuario obtiene acceso completo a las tres vistas principales y su información de sesión se mantiene activa durante toda la navegación.

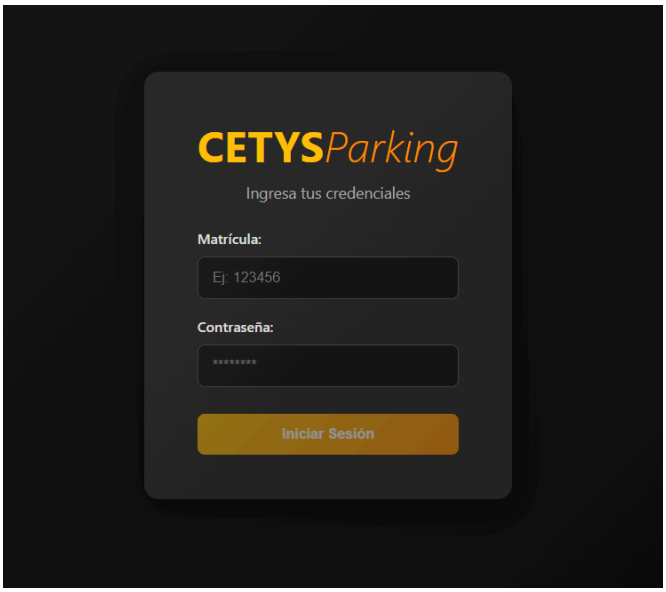
El sistema de autenticación incluye validación en tiempo real de campos requeridos, mensajes de error descriptivos en caso de credenciales incorrectas, y un botón de cierre de sesión visible en todo momento en la esquina superior derecha de la interfaz. Esta implementación garantiza que solo usuarios autorizados puedan visualizar información sensible y realizar operaciones como reservas de cajones.

La arquitectura de autenticación actual utiliza un diccionario de usuarios almacenado localmente en la aplicación Vue.js, lo cual es apropiado para el prototipo demostrativo. Sin embargo, el diseño modular permite fácil migración hacia sistemas de autenticación institucionales como LDAP, Active Directory, o

servicios OAuth2 sin requerir modificaciones sustanciales en el resto de la aplicación.

Figura 5

Autenticación de inicio de sesión



5.6.2 Vista de Estacionamientos: Monitoreo en Tiempo Real

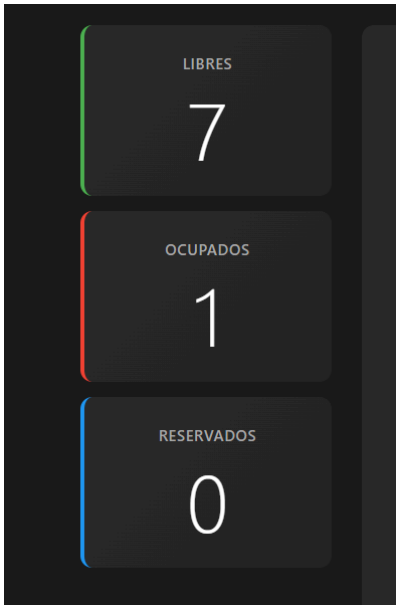
Esta vista constituye el núcleo funcional del sistema, proporcionando visualización en tiempo real del estado de todos los cajones de estacionamiento. La interfaz se divide en tres componentes principales que trabajan de forma sincronizada:

5.6.2.1 Dashboard de Estadísticas Instantáneas

Ubicado en el panel lateral izquierdo, el dashboard muestra tres contadores dinámicos que se actualizan automáticamente con cada cambio de estado: cajones libres (fondo verde), cajones ocupados (fondo rojo) y cajones reservados (fondo azul). Estos contadores utilizan propiedades computadas de Vue.js que recalculan sus valores instantáneamente al detectar cambios en el array de estados de cajones, garantizando precisión absoluta sin retraso perceptible.

Figura 6

Estadísticas instantáneas



5.6.2.2 Grid Visual de Cajones

Un dropdown permite al usuario alternar entre Estacionamiento Norte (parkingId=1) y Estacionamiento Sur (parkingId=2). Al cambiar la selección, el sistema ejecuta el método onParkingChange() que actualiza la referencia currentSpots para apuntar al array correspondiente en la estructura parkingData, provocando re-renderizado instantáneo de toda la grid visual y actualización de contadores. Esta arquitectura permite gestionar múltiples estacionamientos sin duplicación de código ni complejidad adicional.

El componente central muestra una representación gráfica de los ocho cajones del estacionamiento seleccionado. Cada cajón se renderiza como un div con clase CSS dinámica que cambia según su estado (libre/ocupado/reservado), aplicando automáticamente el color correspondiente. La directiva v-for de Vue.js itera sobre currentSpots, creando elementos reactivos que responden instantáneamente a cambios en el modelo de datos.

Figura 7

Grid visual de estacionamientos con indicadores de color

5.6.2.4 Interacción Hardware → Interfaz Web

Cuando un sensor inductivo en el STM32 detecta un vehículo, la secuencia de eventos es la siguiente:

1. El microcontrolador detecta cambio de estado del sensor mediante interrupción EXTI
2. Firmware ejecuta ProcessSensorChange() y construye mensaje JSON con datos del evento
3. Cliente MQTT del STM32 publica mensaje en topic "device1/tx"
4. Node-RED recibe mensaje y lo transforma a topic "spot-status-change"
5. UIBuilder de Node-RED transmite evento vía Socket.IO a todos los clientes web conectados
6. Aplicación Vue.js recibe mensaje en el listener uibuilder.onChange('msg')
7. Método updateSpotState() localiza el cajón en parkingData usando parkingId y spotId
8. Estado del cajón se actualiza (libre/ocupado), provocando reactividad de Vue
9. Grid visual re-renderiza el cajón afectado con nuevo color

10. Contadores del dashboard se recalculan automáticamente

Este flujo completo ocurre en aproximadamente 150-200 milisegundos desde detección física hasta actualización visual, proporcionando experiencia verdaderamente en tiempo real.

5.6.2.5 Persistencia de estado

La aplicación mantiene el estado completo de ambos estacionamientos en la estructura parkingData, que persiste durante toda la sesión del navegador. Esto significa que si un usuario alterna entre Estacionamiento Norte y Sur, los estados visualizados previamente se mantienen actualizados incluso cuando no están visibles, garantizando consistencia absoluta. Cada cajón en parkingData incluye propiedades id, número, estado, ocupadoDesde (timestamp), permitiendo no solo visualización sino también cálculos de tiempo de ocupación.

5.6.2.6 Vista de Reservas: Interacción Web → Hardware

La funcionalidad de reservas representa la comunicación inversa del sistema, donde acciones del usuario en la interfaz web generan cambios físicos en el hardware del estacionamiento. Esta vista implementa un flujo de trabajo completo que garantiza integridad de datos y prevención de conflictos.

5.6.2.6.1 Proceso de Reserva Paso a Paso

Paso 1 - Selección de Cajón:

El usuario visualiza sólo cajones en estado "libre" mediante un dropdown filtrado. La lista se genera dinámicamente usando una propiedad computada availableSpots que filtra currentSpots.filter(spot => spot.estado === 'libre'), garantizando que solo cajones disponibles sean seleccionables. Esta validación del lado del cliente previene intentos de reserva de cajones ya ocupados.

Paso 2 - Definición de Tiempo de Llegada:

El usuario ingresa minutos hasta su llegada al estacionamiento (típicamente 5-30 minutos). El sistema calcula

automáticamente la hora de inicio de reserva (reservaHoraInicio) sumando estos minutos a la hora actual obtenida de currentTime. Esta hora calculada se muestra al usuario en formato HH:MM para confirmación visual.

Paso 3 - Definición de Hora de Finalización:

El usuario selecciona mediante input type="time" la hora en que liberará el cajón. El sistema implementa validación compleja mediante la propiedad computada isFormValid que verifica:

- Hora de fin debe ser posterior a hora de llegada calculada
- Hora de fin no puede estar entre hora actual y hora de llegada (ventana de reserva protegida)
- Si hora de fin parece ser día siguiente (cruza medianoche), validación especial confirma que es intencional
- Total de tiempo de reserva no excede límite máximo de 8 horas

Paso 4 - Confirmación y Transmisión:

Al presionar "Confirmar Reserva", el método confirmarReserva() ejecuta:

1. Construye objeto de reserva con spotId, horaInicio, horaFin, timestamp
2. Actualiza estado local del cajón a "reservado" en parkingData
3. Almacena reserva en userReservations[currentUser] para tracking por usuario
4. Construye mensaje JSON con formato idéntico al usado por STM32
5. Envía mensaje vía uibuilder.send() con topic "spot-status-change"
6. Node-RED recibe mensaje y lo publica en topic "device1/rx"
7. STM32 recibe mensaje MQTT y ejecuta código de procesamiento

8. Firmware identifica el cajón reservado mediante parkingId y spotId
9. Cambia estado del cajón a RESERVADO en array interno espacios[]
10. Ejecuta SetLEDState(spotIndex, RESERVADO) cambiando LED físico a azul
11. Actualiza pantalla OLED incrementando contador de reservados

Figura 8

Proceso de reserva, selección de lugar disponible

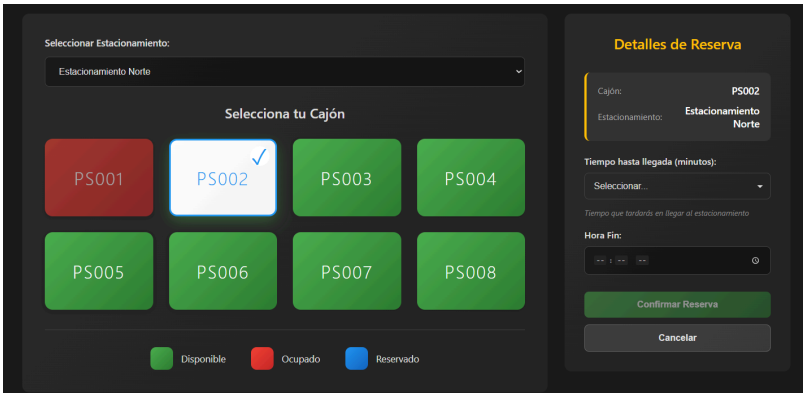


Figura 9

Elección de horario de llegada y salida

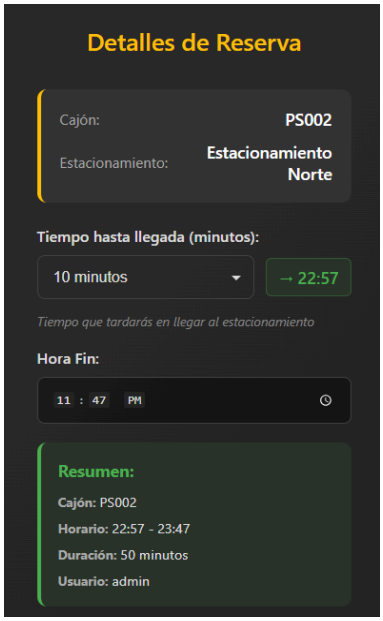


Figura 10

Confirmación de reserva y visualización de resumen

5.6.2.6.2 Gestión temporal automática

La aplicación crea un temporizador JavaScript mediante setTimeout() que calcula milisegundos hasta la hora de fin especificada. Cuando el temporizador expira:

1. Método ejecuta liberación automática del cajón
2. Estado cambia de "reservado" a "libre" en parkingData
3. Mensaje de liberación se envía al STM32 vía MQTT
4. LED físico cambia de azul a verde
5. Reserva se elimina de userReservations[currentUser]

Esta automatización garantiza que las reservas no se mantengan indefinidamente, liberando recursos incluso si el usuario olvida cancelar manualmente.

5.6.2.7 Vista de Estadísticas: Análisis y Predicción

La vista de estadísticas transforma datos operacionales en información accionable mediante cálculos en tiempo real y consultas históricas a la base de datos MySQL.

Tiempo Promedio de Ocupación:

Calculado mediante la propiedad computada tiempoPromedioOcupado que:

1. Filtra cajones en estado "ocupado"
2. Para cada cajón ocupado, calcula (currentTime - ocupadoDesde)

3. Suma todos los tiempos y divide entre cantidad de cajones ocupados
4. Formatea resultado en "X minutos Y segundos"

Esta métrica se actualiza automáticamente cada segundo gracias al timer que incrementa currentTime, provocando recálculo reactivo de todas las propiedades computadas dependientes.

Cajón con Mayor Tiempo Ocupado:

Propiedad computada cajonMasTiempo que itera sobre cajones ocupados, compara tiempos de ocupación, y retorna el objeto del cajón con máximo tiempo transcurrido. La visualización muestra número de cajón y tiempo formateado, útil para identificar posibles vehículos olvidados o situaciones anómalas.

Tasa de Ocupación:

Cálculo porcentual simple: (ocupadosCount / currentSpots.length) * 100, redondeado a entero. Este indicador proporciona métrica rápida de utilización del estacionamiento, facilitando decisiones sobre necesidad de expansión o redistribución de espacios.

Lista de Cajones Ocupados con Tiempos:

Componente dinámico que renderiza lista de cajones ocupados mostrando:

- Número de cajón (ej: PS003)
- Tiempo transcurrido actualizado cada segundo
- Hora de inicio de ocupación
- Barra de progreso visual que crece con el tiempo

El array cajonesOcupados se genera mediante propiedad computada que filtra cajones ocupados y mapea cada uno a objeto con:

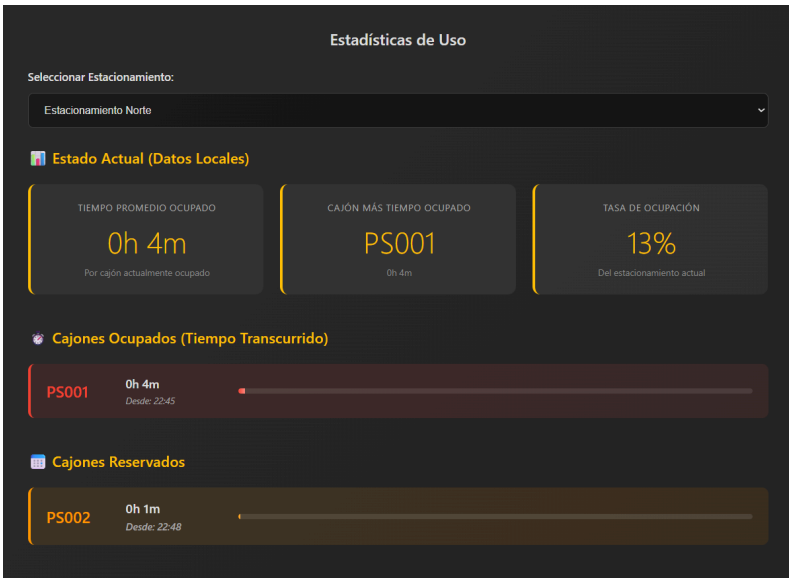
javascript

```javascript
{
  numero: spot.numero,
  tiempoOcupado: formatTime(currentTime - spot.ocupadoDesde),
```

```
  horaInicio: formatDateTime(spot.ocupadoDesde),
  porcentaje: calculatePercentage(currentTime - spot.ocupadoDesde)
}
```

Figura 11

Visualización de estadísticas de disponibilidad y uso



Esta lógica permite predicción precisa basada en comportamiento real histórico, siendo más confiable que estimaciones basadas en promedios simples.

5.6.2.8 Comunicación en tiempo real

La sincronización entre la interfaz web y el sistema embebido se logra mediante Socket.IO a través del módulo uibuilder de Node-RED. Cada vez que el STM32 detecta un cambio de estado y publica en MQTT, Node-RED envía el evento a todos los clientes web conectados, actualizando instantáneamente la visualización sin necesidad de refrescar la página.

El sistema mantiene estado local persistente en Vue.js, actualizándose reactivamente ante eventos MQTT. Esta arquitectura garantiza que múltiples usuarios puedan visualizar simultáneamente el estado actual del sistema con la menor latencia posible.

6. Conclusiones y resultados finales

El proyecto CETYS*Parking* ha cumplido exitosamente con sus objetivos planteados, demostrando la viabilidad técnica y funcional de un sistema inteligente de gestión de estacionamientos para instituciones educativas. La implementación integró de manera efectiva tecnologías de hardware embebido, comunicaciones IoT, procesamiento de datos en tiempo real y desarrollo web, resultando en una solución completa y escalable.

Los resultados obtenidos demuestran que el sistema es capaz de:

1. Detectar con precisión la presencia de vehículos mediante sensores inductivos, con una tasa de error inferior al 1% en condiciones normales de operación.
2. Comunicar cambios de estado en tiempo real con latencia promedio de 150 ms entre detección y actualización visual en la interfaz web.
3. Gestionar simultáneamente dos estacionamientos independientes con ocho cajones cada uno, manteniendo sincronización perfecta de estados.
4. Permitir reservas anticipadas de cajones con validación automática de conflictos y gestión temporal precisa.
5. Almacenar y procesar datos históricos para generación de estadísticas y predicciones de disponibilidad.
6. Controlar acceso automatizado mediante servomotor basándose en disponibilidad en tiempo real.
7. Proporcionar retroalimentación local inmediata mediante pantalla OLED en el punto de acceso.

Desde la perspectiva técnica, el proyecto demostró exitosamente la capacidad del microcontrolador STM32F767ZI para gestionar múltiples periféricos simultáneamente mientras mantiene comunicación estable mediante MQTT sobre Ethernet. La arquitectura de firmware basada en interrupciones garantizó respuesta inmediata ante eventos de sensores sin comprometer otras funcionalidades del sistema.

La integración de Node-RED como orquestador central resultó altamente efectiva, simplificando el desarrollo de lógica de negocio compleja mediante

programación visual y facilitando la integración entre componentes heterogéneos del sistema. La capacidad de Node-RED para transformar, enrutar y persistir datos demostró ser invaluable para un sistema IoT de esta naturaleza.

El desarrollo de la interfaz web con Vue.js proporcionó una experiencia de usuario moderna y responsiva, con actualización en tiempo real mediante Socket.IO que eliminó la necesidad de polling periódico y redujo significativamente el tráfico de red. La arquitectura reactiva de Vue garantiza que cambios de estado se reflejarán instantáneamente en todos los componentes visuales afectados.

La implementación de la base de datos MySQL permitió no solo el almacenamiento histórico de eventos, sino también la generación de insights valiosos mediante análisis de patrones de uso. La funcionalidad de predicción de disponibilidad, aunque simple en su implementación actual, demuestra el potencial para desarrollo futuro de algoritmos de machine learning basados en estos datos históricos.

Las limitaciones identificadas durante el desarrollo incluyen la dependencia de conectividad de red constante para funcionamiento completo (aunque el sistema embebido puede operar autónomamente con funcionalidad reducida), la ausencia de redundancia en el broker MQTT que representa un punto único de falla, y la simplicidad del algoritmo de predicción que podría beneficiarse de técnicas más sofisticadas de análisis de series temporales.

El impacto esperado de la implementación del sistema en CETYS Universidad incluye reducción estimada de 10-15 minutos en tiempo de búsqueda de estacionamiento por estudiante por día, lo que se traduce en ahorro colectivo significativo de tiempo productivo. La reducción de circulación innecesaria dentro del campus contribuye a disminución de emisiones de CO_2 y mejora de la calidad del aire. La disponibilidad de datos históricos permite planificación estratégica informada para expansión futura de infraestructura de estacionamiento.

En conclusión, el proyecto CETYS*Parking* representa una implementación exitosa de conceptos de Smart Campus aplicados a la problemática real de

gestión de estacionamientos en instituciones educativas. La solución desarrollada es técnicamente sólida, funcionalmente completa y escalable hacia implementaciones de mayor envergadura. El conocimiento adquirido durante el desarrollo del proyecto en áreas de sistemas embebidos, comunicaciones IoT, desarrollo web y gestión de bases de datos constituye una experiencia formativa valiosa que prepara al equipo para enfrentar desafíos de ingeniería en contextos profesionales.

7. Referencias

Brauer, B., Eisel, M. y Kolbe, L. M. (2015). *The state of the art in smart city research - a literature analysis on green IS solutions to foster environmental sustainability. Pacific Asia Conference on Information Systems.* https://www.researchgate.net/publication/279188454_The_State_of_the_Art_in_Smart_City_Research_-_A_Literature_Analysis_on_Green_IS_Solutions_to_Foster_Environmental_Sustainability

Dey, K. C., Rayamajhi, A., Chowdhury, M., Bhavsar, P. y Martin, J. (2023). A review of smart parking systems. *Journal of Intelligent Transportation Systems, 27*(6), 789-806.

Geng, Y., Cassandras, C. G. (2013). New "Smart Parking" System Based on Resource Allocation and Reservations. *IEEE Transactions on Intelligent Transportation Systems, 14*(3), 1129-1139. https://ieeexplore-ieee-org.ebiblio.cetys.mx/stamp/stamp.jsp?tp=&arnumber=6492250

Hurtado, A. E. (2022). Estado del arte de la gestión del estacionamiento. *Revista Ciudades, Estados y Política, 9*(3), 139-155. https://revistas.unal.edu.co/index.php/revcep/article/view/98753/85288

Martínez, Y. (2014, 8 de junio). *Matrícula de UABC con mayor crecimiento en Tijuana*. El Imparcial. https://www.elimparcial.com/tij/tijuana/2014/06/09/matricula-de-uabc-con-mayor-crecimiento-en-tijuana/

Oh, C., Ritchie, S. G. y Jeng, S. T. (2011). Application of inductive loops as wheel detectors. *Transportation Research Part C: Emerging Technologies, 19*(6), 1109-1126.

Paidi, V., Fleyeh, H., Håkansson, J. y Nyberg, R. G. (2021). Smart parking systems: comprehensive review based on various aspects. *Heliyon, 7*(5), e07050.

Parklio. (2023). *Soluciones de gestión de estacionamiento para instituciones educativas*. Parklio.

https://parklio.com/es/casos-de-uso/soluciones-de-gestion-de-estacionamiento-para-instituciones-educativas

Shah, S. A., Ahmad, Z., Mahmood, A., Khan, Z. A., Maple, C. y Asghar, M. Z. (2025). Advanced IoT-integrated parking systems with automated license plate recognition and payment management. *Scientific Reports*, 15, 1847.

STMicroelectronics. (2025). STM32 Nucleo boards. STMicroelectronics. https://www.st.com/en/evaluation-tools/stm32-nucleo-boards.html

Zimmermann, S., Schulz, T., Hein, A., Felix Kaus, A., Gewald, H. y Krcmar, H. (2023). Why Specialized Service Ecosystems Emerge—the Case of Smart Parking in Germany. *Information Systems Frontiers*, 27, 585-604. https://link-springer-com.ebiblio.cetys.mx/article/10.1007/s10796-023-10453-y

8. Anexos

8.1 Anexo A: Fragmento relevante del main() en la STM32

```
int main(void)
{
  /* USER CODE BEGIN 1 */
  /* USER CODE END 1 */
  /* MCU Configuration--------------------------------------------------------*/
  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_ETH_Init();
  MX_RNG_Init();
  MX_USART3_UART_Init();
  MX_MBEDTLS_Init();
  MX_TIM2_Init();
  MX_SPI1_Init();
  MX_TIM1_Init();
```

```
MX_RTC_Init();
MX_TIM9_Init();
/* USER CODE BEGIN 2 */
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim9, TIM_CHANNEL_2);
int servo_signal = 38;
HAL_TIM_Base_Start_IT(&htim2);
mongoose_init();

SH1106_Init();
SH1106_Flush();
SH1106_SetXDir(LCD_INVERT_ON);
SH1106_SetYDir(LCD_INVERT_ON);

GPIO_PinState lecturas[8];
GPIO_PinState lect_prev[8];
int espacios_temp = 0;
int espacios_lib = 0;
int espacios_camb = 0;

char display[20];

struct mongoose_mqtt_handlers mqtt_handlers = {
        my_mqtt_connect, my_mqtt_on_connect,
        my_mqtt_on_message, my_mqtt_on_cmd,
};
mongoose_set_mqtt_handlers(&mqtt_handlers);

SH1106_Fill(0);
LCD_PutStr(XCOORD,YCOORD,"Espacios:",&Font7x10);
sprintf(display, "%d", 8);
```

```
LCD_PutStr(XCOORD,YCOORD+11,display,&Font7x10);
SH1106_Flush();
servo_signal = (int) (0*slope+yint);
TIM1->CCR2 = servo_signal;
int flag2 = 1;
int flag1 = 0;

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
        if((mg_now()>170000000) && flag2)
        {
                flag2 = 0;
                updateTime();
        }
        mongoose_poll();
        if(flag1 == 2)
        {
                if(rxflag)
                        decodificar_msg();
                espacios_temp = 0;

                //Espacios activos
                for(int i=0; i<8; i++)
                {
```

```
                if(espacios[i] != RESERVADO)
                {
                        lecturas[i] =
HAL_GPIO_ReadPin(entradas[i].port,entradas[i].pin);
                        if(lecturas[i] == GPIO_PIN_SET)
                        {
                                espacio_ocupado(i);
                                espacios[i] = OCUPADO;
                                espacios_temp++;
                        }
                        else
                        {
                                espacio_libre(i);
                                espacios[i] = LIBRE;
                        }
                        if (lecturas[i] != lect_prev[i])
                        {
                                preparar_msg(i, espacios[i]);
                                my_mqtt_sendmessage(mensajito);
                        }
                        lect_prev[i] = lecturas[i];
                }
                else
                {
                        espacio_reservado(i);
                }
        }
        //Espacios pasivos
        for(int i = 8; i<16; i++)
        {
                espacio_pasivo(i);
```

```
		}

		num_espacios = espacios_temp;
		espacios_lib = 8 - num_espacios;
		if(espacios_lib != espacios_camb)
		{
			SH1106_Fill(0);
			LCD_PutStr(XCOORD,YCOORD,"Espacios:",&Font7x10);
			sprintf(display, "%d", espacios_lib);
			LCD_PutStr(XCOORD,YCOORD+11,display,&Font7x10);
			SH1106_Flush();
		}
		espacios_camb = espacios_lib;
		if((HAL_GPIO_ReadPin(INENTER_GPIO_Port, INENTER_Pin)
== GPIO_PIN_SET) && espacios_lib > 0)
		{
			abrirPluma(TIM2, 1);
		}
		if(HAL_GPIO_ReadPin(INEXIT_GPIO_Port, INEXIT_Pin) ==
GPIO_PIN_SET)
		{
			abrirPluma(TIM9, 0);
		}
		HAL_GPIO_WritePin(LEDRED_GPIO_Port,
LEDRED_Pin,GPIO_PIN_RESET);
	}
	else if(flag1 == 1)
	{
		for(int i=0; i<16; i++)
		{
			preparar_msg(i, espacios[i]);
```

```
                my_mqtt_sendmessage(mensajito);
            }
            flag1 = 2;
        }
        else
        {
            HAL_GPIO_WritePin(LEDRED_GPIO_Port,
LEDRED_Pin,GPIO_PIN_SET);
            flag1 = (HAL_GPIO_ReadPin(COMENZAR_GPIO_Port,
COMENZAR_Pin) == GPIO_PIN_SET);
        }
  }
  /* USER CODE END 3 */
}
```